Latest updates: https://dl.acm.org/doi/10.1145/3690624.3709335

RESEARCH-ARTICLE

Collaboration of Large Language Models and Small Recommendation Models for Device-Cloud Recommendation

ZHEQI LV, Zhejiang University, Hangzhou, Zhejiang, China

TIANYU ZHAN, Zhejiang University, Hangzhou, Zhejiang, China

WENJIE WANG, National University of Singapore, Singapore City, Singapore

XINYU LIN, National University of Singapore, Singapore City, Singapore

SHENGYU ZHANG, Zhejiang University, Hangzhou, Zhejiang, China

WENQIAO ZHANG, Zhejiang University, Hangzhou, Zhejiang, China

View all

Open Access Support provided by:

Zhejiang University

National University of Singapore

PDF Download
3690624.3709335.pdf
16 December 2025
Total Citations: 3
Total Downloads: 213

Published: 20 July 2025

Citation in BibTeX format

KDD '25: The 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining
August 3 - 7, 2025
Toronto ON, Canada

Conference Sponsors:
SIGMOD
SIGKDD

KDD '25: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (July 2025)
https://doi.org/10.1145/3690624.3709335
ISBN: 9798400712456

Collaboration of Large Language Models and Small Recommendation Models for Device-Cloud Recommendation

Zheqi Lv
Zhejiang University
Hangzhou, China
zheqilv@zju.edu.cn

Tianyu Zhan
Zhejiang University
Hangzhou, China
yuzt@zju.edu.cn

Wenjie Wang*
National University of Singapore
Singapore, Singapore
wenjiewang96@gmail.com

Xinyu Lin
National University of Singapore
Singapore, Singapore
xylin1028@gmail.com

Shengyu Zhang*
Zhejiang University
Hangzhou, China
sy_zhang@zju.edu.cn

Wenqiao Zhang
Zhejiang University
Hangzhou, China
wenqiaozhang@zju.edu.cn

Jiwei Li
Zhejiang University
Hangzhou, China
jiwei_li@zju.edu.cn

Kun Kuang*
Zhejiang University
Hangzhou, China
kunkuang@zju.edu.cn

Fei Wu
Zhejiang University
Hangzhou, China
wufei@zju.edu.cn

Abstract

Large Language Models (LLMs) for Recommendation (LLM4Rec) is a promising research direction that has demonstrated exceptional performance in this field. However, its inability to capture real-time user preferences greatly limits the practical application of LLM4Rec because (i) LLMs are costly to train and infer frequently, and (ii) LLMs struggle to access real-time data (its large number of parameters poses an obstacle to deployment on devices). Fortunately, small recommendation models (SRMs) can effectively supplement these shortcomings of LLM4Rec diagrams by consuming minimal resources for frequent training and inference, and by conveniently accessing real-time data on devices.

In light of this, we designed the *Device-Cloud **L**LM-**S**RM **C**ollaborative **Rec**ommendation Framework* (LSC4Rec) under a device-cloud collaboration setting. LSC4Rec aims to integrate the advantages of both LLMs and SRMs, as well as the benefits of cloud and edge computing, achieving a complementary synergy. We enhance the practicability of LSC4Rec by designing three strategies: collaborative training, collaborative inference, and intelligent request. During training, LLM generates candidate lists to enhance the ranking ability of SRM in collaborative scenarios and enables SRM to update adaptively to capture real-time user interests. During inference, LLM and SRM are deployed on the cloud and on the device, respectively. LLM generates candidate lists and initial ranking results based on user behavior, and SRM get reranking results based on the candidate list, with final results integrating both LLM's and SRM's scores. The device determines whether a new candidate list is needed by comparing the consistency of the LLM's and SRM's sorted lists. Our comprehensive and extensive experimental analysis validates the effectiveness of each strategy in LSC4Rec.

CCS Concepts

• **Computing methodologies** → **Cooperation and coordination**; • **Information systems** → **Retrieval models and ranking**.

Keywords

Sequential Recommendation, Device-Cloud Collaboration, Large Language Model

ACM Reference Format:
Zheqi Lv, Tianyu Zhan, Wenjie Wang, Xinyu Lin, Shengyu Zhang, Wenqiao Zhang, Jiwei Li, Kun Kuang, and Fei Wu. 2025. Collaboration of Large Language Models and Small Recommendation Models for Device-Cloud Recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3690624.3709335

1 Introduction

Traditional sequential recommendation models, such as DIN [64], GRU4Rec [12], SASRec [15], and so on, have achieved great success in both academia and industry. Here, we refer to these models that do not use large language models (LLMs) as their backbone and do not integrate user behavior into textual prompts as input for recommendation as small recommendation models (SRMs). Currently, LLMs have been explored for recommendation scenarios (LLM4Rec) [11, 45, 61], demonstrating stronger multi-task generalization capabilities and superior single-task performance, which position LLM4Rec as a highly promising research direction.

However, LLMs face challenges in capturing real-time user preferences for two reasons: (i) **Expensive to train and inference frequently.** LLMs are built with an immense number of parameters [1, 11, 42], posing difficulties for the efficiency of LLM's training

*Corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '25, Toronto, ON, Canada
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1245-6/25/08
https://doi.org/10.1145/3690624.3709335

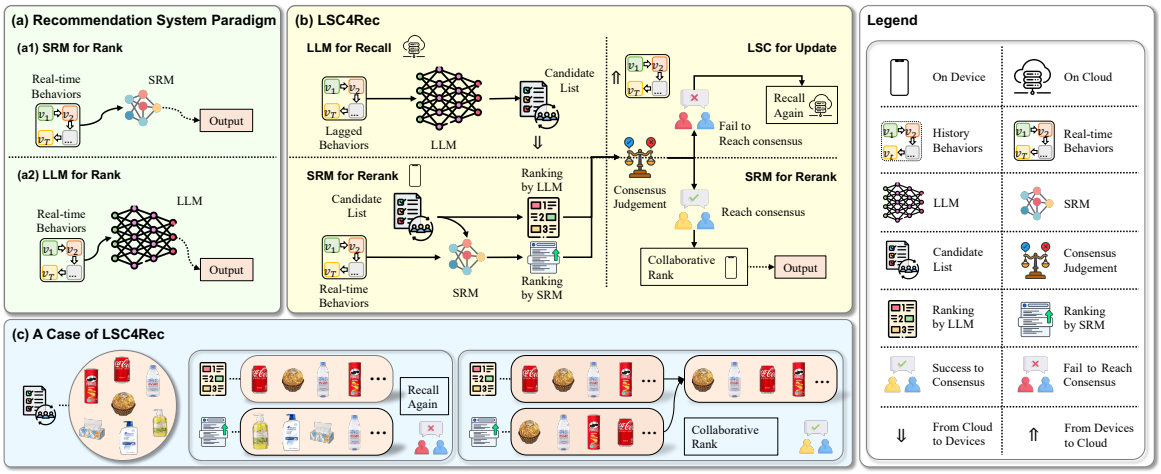

Figure 1: (a) respectively describes the conventional SRM4Rec and LLM4Rec diagrams. (b) is an overview of the proposed LSC4Rec, a device-cloud collaboration framework, which includes a collaborative training strategy, a collaborative inference strategy, and a collaborative update strategy. (c) is a qualitative case of our LSC4Rec framework, showing effective collaborative inference and collaborative-decision request.

and inference. Moreover, the rapid increase of user behavior data exacerbates the situation, making frequent training of LLMs impractical. (ii) **Difficult to access real-time user behavior data.** In practical applications, user real-time behaviors are generated on devices while trainable LLMs are typically deployed on the cloud due to computational resource considerations [33–35, 48, 49]. Transmitting tremendous user data to the cloud in real time could cause non-trivial communication costs and latency.

As a counterpart, SRMs could potentially supplement the shortcomings faced by LLM4Rec. (i) **SRM can be trained and inferred frequently at a significantly lower cost.** The smaller number of parameters and simpler model structure of SRM make its training and inference less resource-intensive, allowing for frequent training and inference for SRM. (ii) **SRM can easily access all real-time data.** The minimal use of storage and computational resources in SRM enables its deployment on devices. Therefore, accessing and handling real-time data could be much more efficient for SRM.

In this work, we propose complementing **L**Lms with **S**Rms for efficient and effective device-cloud **C**ollaborative **Rec**ommendation, namely LSC4Rec. The basic intuition of LSC4Rec lies in the *dual optimization* of on-cloud LLMs for item candidate generation and on-device SRMs for item reranking. In a recommendation session, on-cloud LLMs leverage the strong user preference capturing ability to generate an item candidate list, on-device SRMs are adapted to encode the user's real-time behaviors and promptly score the items in the candidate list for real-time reranking. In such a device-cloud framework, LSC4Rec can reduce the training and inference frequency of on-cloud LLMs by using on-device SRMs to capture users' real-time preferences. Meanwhile, LSC4Rec can diminish the communication costs between the cloud and devices.

However, it is non-trivial to instantiate LSC4Rec due to the following issues:

- How can the SRM contribute to the LSC4Rec framework? Simply training the SRM and LLM separately and then deploying them in the LSC4Rec framework is empirically found ineffective, where SRM struggles to effectively rerank the item candidates from LLMs. Due to the prior knowledge of LLMs, their generated item candidates have a different distribution of negative samples from that of randomly sampled negative ones in SRM. Therefore, SRM cannot accurately distinguish the positive and negative sampled generated by the LLM during inference (Refer to empirical evidence in Table 4).
- How to fully utilize the capabilities of the LLM during the inference stage? In the inference stage, after the LLM generates a candidate list, only using the SRM to score items in the candidate list based on real-time data is equivalent to discarding part of the LLM's powerful generalization ability. This results in limited effectiveness of the LLM during inference, making it difficult to provide continuous benefits to the SRM's ranking.
- How to determine the shift of user preferences and update user behaviors from devices to the cloud? In recommendation scenarios, user preferences are shifting over time. Therefore, it is essential to identify the user preference shifts and promptly update user behaviors from the device to the cloud, thereby obtaining a new candidate list from the LLM that better matches the user's latest preferences.

To address the aforementioned challenges of LSC4Rec, as shown in Figure 1, we designed collaborative training, collaborative inference, and collaborative-decision request. Specifically,

- Collaborative Training. Both LLM and SRM are first pre-trained based on historical data separately. Thereafter, we segment and combine items in the candidate list generated by LLM and feed this enhanced data to SRM to improve its ability to rank the candidate list retrieved by LLM. At the same time, we allow SRM to undergo adaptive training to adapt to the data distribution on the device, further enhancing its ability to extract real-time user interests more effectively.
- Collaborative Inference. LLM infers a list of candidate items and their ranking based on user behavior, which may come from a pool of millions or even billions of items. Then, SRM

extracts user interests from real-time behavior to rerank the candidate items and obtain a ranking, and the reranking results of SRM are integrated with the initial ranking results of LLM through our designed score integration strategy to produce collaborative inference results. The score integration strategy includes normalizing the scores of LLM and SRM to the same range, score filtering, and weight allocation.

- Collaborative-Decision Request. It determines the necessity of requesting a new candidate list from LLM by comparing the similarity between these two sets of interests. Before outputting the results, LSC4Rec needs to compare the inconsistency of the initial ranking results based on LLM with the reranking results based on SRM. If there is low inconsistency (i.e., similar ranking results), the collaborative inference result is output. Otherwise, real-time behavior is transmitted to the LLM in the cloud to generate a new candidate list and ranking results based on short-term interests.

In summary, our contributions include four aspects:

- We concentrate on a promising but under-explored research direction, specifically the challenges faced by LLMs in capturing user preferences in real-time during practical applications.
- We propose a novel collaborative recommendation framework called LSC4Rec. LSC4Rec effectively combines the strengths of LLMs and SRMs in a device-cloud system, paving the way for a promising direction in future research.
- We design collaborative training, collaborative inference, and collaborative-decision request to address the three challenges of LSC4Rec and make it more practical.
- We have conducted comprehensive and extensive experiments on various LLMs and SRMs across multiple datasets. Results show the effectiveness of LSC4Rec.

2 Related Work

2.1 LLM for Recommendation

LLMs are increasingly prominent in natural language processing [16, 18, 21, 25, 26, 40, 43, 46, 50, 65, 66], driving research in their application for recommendation systems [6, 11, 17, 22, 23, 31, 52, 57, 62]. The advent of generative models like GPT transformed LLM-based recommendation into generative processes, treating recommendations as natural language tasks [44]. Early methods relied on prompting [10, 39] or in-context learning [24] but often fell short of task-specific models. Recent advancements involve fine-tuning LLMs for better alignment with recommendation tasks. P5 [11] introduced a unified framework for fine-tuning FLAN-T5 [36] across various recommendation tasks. InstructRec [53] and TALLRec [1] further adapted FLAN-T5 and LLaMA models, respectively, for recommendation tasks using instruction tuning. GenRec's [13] approach involves direct instruction tuning on the LLaMA model for the generative recommendation, showcasing the evolving strategies in integrating LLMs with recommendation systems. P5 [11] and POD [17] are two large recommendation models based on the T5, aiming to unify the multiple recommendation tasks. They transform the recommendation task into textual format and use the text as input to the LLM to obtain recommendation results. LLMs possess stronger generalization capabilities compared to SRMs but face challenges in terms of high costs and deployment, making it difficult to integrate real-time user preferences. The aforementioned studies have not addressed this issue.

2.2 Sequential Recommendation

Sequential recommendation algorithms model users' behavior sequences and predict their next actions. They have been widely applied in scenarios such as e-commerce, short video recommendations, education, and healthcare. Classic recommendation models such as Caser [41], GRU4Rec [12], DIN [64], and SASRec [15] are still commonly used today. In recent years, many studies have focused on different aspects of sequential recommendation, such as personalization [32–34, 38], multimodality [14, 51, 54, 55], privacy protection [20], user feedback [9, 56], cross-domain [27, 63], cold start [4, 28], long-tail [59], session-based [37], debias [3], structure learning [7], disentanglement [5], LLM-based [2, 58], generative-based [23, 60], fairness [47], etc. Although Sequential Recommendation Models (SRMs) require fewer hardware resources and are easier to deploy on devices, they have limitations in feature extraction and generalization capabilities. These studies, however, have not addressed the deployment of SRMs on devices or the associated limitations.

2.3 Device-Cloud Recommendation

In more realistic recommendation scenarios, the collaboration of device intelligence and cloud intelligence is often required [8, 19, 29, 30, 33–35, 48, 49]. For instance, DCCL [49] and MetaController [48] made early attempts at device-cloud collaborative recommendation models. DUET [34] and IntellectReq [33] explored ways to help device models overcome the generalization limitations imposed by their model size. Their specific approach involves having the device model handle personalization, while the cloud model generates the necessary parameters for the device model based on data from the device. However, when LLMs are used as cloud models, they introduce greater cloud computing loads and pose challenges in directly collaborating with SRMs. These studies do not address device-cloud collaborative recommendation strategies when LLMs serve as the cloud model.

3 Methodology

3.1 Problem Formulation and Notation

3.1.1 Data. We use $\mathcal{X} = \{u, v, s\}$ to represent a piece of data, and $\mathcal{Y} = \{y\}$ to represent the corresponding label. Specifically, u, v, s represent user ID, item ID and user's click sequence respectively. Historical data collected for a period of time from devices is represented by $\mathcal{D}_H = \{\mathcal{X}_H^{(i)}, \mathcal{Y}_H^{(i)}\}_{i=1}^{N_H}$. The data augmented based on $\mathcal{D}_H$ is denoted as $\mathcal{D}_{H-\text{Aug}}$. In the inference stage, users will generate real-time behaviors on the devices. These real-time behavioral data are represented as $\mathcal{D}_R = \{\mathcal{X}_R^{(i)}, \mathcal{Y}_R^{(i)}\}_{i=1}^{N_R}$. The lagged behavioral data are represented as $\mathcal{D}_R^{'}$. $\mathcal{D}_R^{'}$ is also data generated by users during the inference stage, and it represents the last set of data uploaded to the cloud by the user. Since the device-to-cloud synchronization is not real-time, there is a time difference between $\mathcal{D}_R^{'}$ and $\mathcal{D}_R$. More precisely, $\mathcal{D}_R^{'}$ lags behind $\mathcal{D}_R$ by Δt time units.

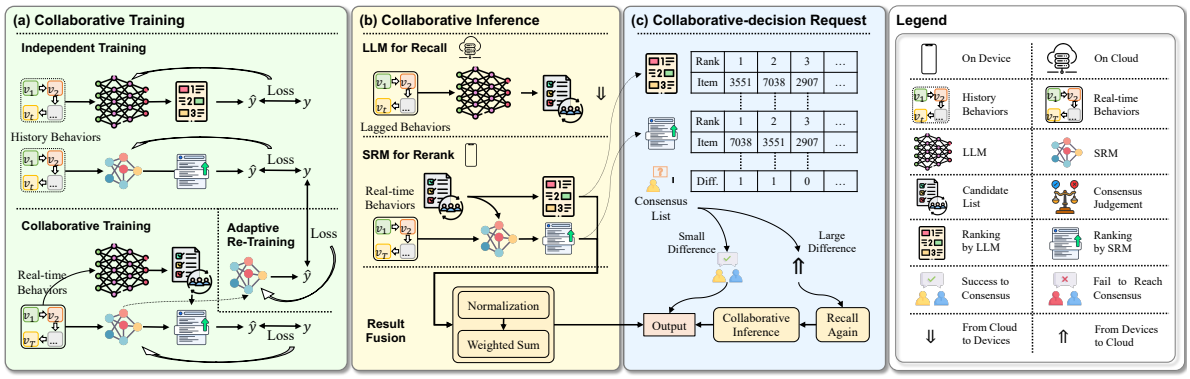

Figure 2: Overview of the LSC4Rec. (a) describes collaborative training, including independent training, cooperative training, and adaptive re-training. (b) describes collaborative inference, including cooperative inference and result fusion. (c) describes collaborative-decision request.

3.1.2 Model. LLM and SRM are denoted as $\mathcal{M}_L$ and $\mathcal{M}_S$, respectively. The output of LLM, that is, the candidate list recalled by LLM based on $\mathcal{D}_H$ or $\mathcal{D}_R$, is denoted as $\mathcal{S} = \{v_i\}_1^{\mathcal{N}_{\text{candidate}}}$. The order of items in this list is considered to be the initial ranking, denoted as $\hat{\mathrm{Y}}_{\text{init}}$. The output of SRM re-ranking based on real-time data is denoted as $\hat{\mathrm{Y}}_{\text{rerank}}$.

3.1.3 Formula. To facilitate the description of the training process, here we use $\phi(\mathcal{D})$ to represent that the model is trained using data $\mathcal{D}$, $\phi_{Re}(\mathcal{D})$ to represent that the model is re-trained using data $\mathcal{D}$, $Aug(\mathcal{D})$ to represent data augmentation based on data $\mathcal{D}$, $\bigoplus$ to represent collaborative algorithms, Π to represent the function that determines the inconsistency or similarity of the sorting of two lists. We formulate our proposed LSC4Rec as follows,

Collaborative Training:

$$\begin{gathered}\mathcal{M}_L = \phi(\mathcal{D}_H), \mathcal{M}_S = \phi(\mathcal{D}_H);\\ \mathcal{M}_S = \phi_{Re}(\{\mathcal{D}_H, \mathcal{D}_{H-\text{Aug}} = Aug(\mathcal{M}_L(\mathcal{D}_H))\});\\ \mathcal{M}_S = \phi(\mathcal{D}_R).\end{gathered} \tag{1}$$

Collaborative Inference:

$$\begin{aligned}\mathcal{S}, \hat{\mathrm{Y}}_{\text{init}} &= \mathcal{M}_L(\mathcal{D}_R^{'});\\ \hat{\mathrm{Y}}_{\text{rerank}} &= \mathcal{M}_S(\mathcal{S}; \mathcal{D}_R);\\ \hat{\mathrm{Y}} &= \hat{\mathrm{Y}}_{\text{init}} \bigoplus \hat{\mathrm{Y}}_{\text{rerank}}.\end{aligned} \tag{2}$$

Collaborative-Decision Request:

$$\text{Request} = \mathbb{1}(\Pi(\hat{\mathrm{Y}}_{\text{init}}, \hat{\mathrm{Y}}_{\text{rerank}}) \geq \text{Threshold}). \tag{3}$$

Note that the function symbols here are rough and are only used to formulate the task description. The details of the formula will be elaborated in subsequent subsections of this section.

3.2 LSC4Rec

This section describes our proposed LSC4Rec. As shown in Figure 2, our LSC4Rec framework includes (a) Collaborative Training, (b) Collaborative Inference, and (c) Collaborative-Decision Request. Collaborative Training aims to enable SRM to provide benefits to the LSC4Rec framework. Collaborative Inference aims to complement the strengths of LLM and SRM during the inference stage, resulting in more accurate recommendation outcomes. Collaborative-Decision Request targets to determine whether there has been a shift in user preferences, in order to decide whether the candidate list needs to be updated.

3.2.1 Retropy of Existing Sequential Recommendation Diagrams. Here we review the paradigm of sequential recommendation.

In the training stage, if we disregard the process of data textualization required for LLM4Rec, the method by which LLM4Rec and SRM4Rec obtain loss is the same, and can be formalized as follows,

$$\mathcal{L} = \sum_{u,v,s,y \in \mathcal{D}_H} l(y, \hat{y} = \mathcal{M}(u, v, s)). \tag{4}$$

In the above equation, l represents loss function, $\mathcal{M}$ generically represents both LLM and SRM. $\mathcal{M}(\cdot)$ denotes the output of the model when $\cdot$ is given as input. During gradient backpropagation, LLM4Rec and SRM4Rec differ slightly. LLM4Rec has the option to update the instruction, prompt, and model, while SRM4Rec typically focuses on updating the model itself.

In the inference stage, if we disregard the process of data textualization required for LLM4Rec, the inference processes of both are essentially consistent.

3.2.2 Collaborative training. Collaborative Training consists of Independent training, Cooperative training, and Adaptive re-training.

`Independent training.` When LLM and SRM are trained separately, the calculation of loss is the same as equation 4. The backpropagation for SRM can be formalized as $\arg\min_{\theta_S} L$. In the above formula, θ_S represents the parameter of $\mathcal{M}_S$. Based on prior knowledge, we fix the instruction and prompt of the LLM and then fine-tune the model's parameters. The backpropagation of LLM can be formalized as $\arg\min_{\theta_L} L$. In the above equation, θ_L represents the parameter of $\mathcal{M}_L$.

`Cooperative training.` After independent training, we input $\mathcal{X}_H$ into the LLM to obtain inference results. For each sample x, we infer $\mathcal{N}_{candidate}$ items, forming the candidate list S. Furthermore, we freely combine items that are difficult for the LLM to distinguish, that is, items with similar tokens, to obtain an enhanced candidate list S_{Aug}. Subsequently, we also input each sample x into the SRM, allowing the SRM to re-rank within the candidate list S_{Aug}. The resulting prediction is denoted as $\hat{y}$. So the loss function can be formulated as,

$$S = \mathcal{M}_L(x), \quad S_{\text{Aug}} = \text{Augment}(S), \tag{5}$$

$$\mathcal{L} = \sum\nolimits_{u,v,s,y \in \mathcal{D}_H} l(y, \hat{y} = \mathcal{M}_S(u, v, s) | S_{\text{Aug}}). \tag{6}$$

We fix the model parameters of LLM so that the gradient generated by loss only backpropagates to SRM, so the optimization function is $\arg\min_{\theta_S} L$.

`Adaptive re-training.` After LLM and SRM are trained, LLM is deployed in the cloud and SRM on the device. However, as mentioned in the Introduction section, due to network bandwidth limitations and delays in data processing on the device side, it is difficult for LLM to access real-time data. Even if it can access such data, its large number of parameters makes it challenging to train and deploy the model quickly. Therefore, SRM needs to be trained based on real-time data on the device, with the optimization goal being $\arg\min_{\theta_S} L$, where the loss function is as follows,

$$\mathcal{L} = \sum\nolimits_{u,v,s,y \in \mathcal{D}_R} l(y, \hat{y} = \mathcal{M}_S(u, v, s)). \tag{7}$$

3.2.3 Collaborative inference. Collaborative Inference consists of Cooperative Inference and Result Fusion.

`Cooperative Inference.` After training is completed, LLM and SRM are deployed separately in the cloud and on the device. Here, we introduce how to merge the initial ranking results $\hat{Y}_{init}$ output by LLM with the re-ranking results $\hat{Y}_{\text{rerank}}$ of SRM during the inference stage.

$$\begin{aligned} \mathcal{S}, \hat{Y}_{\text{init}} &= \mathcal{M}_L(\mathcal{D}_R^{'}); \\ \hat{Y}_{\text{rerank}} &= \mathcal{M}_S(\mathcal{S}; \mathcal{D}_R). \end{aligned} \tag{8}$$

`Result Fusion.` LLM and SRM have different ranking principles; LLM is generative while SRM is non-generative, leading to different ranking scales for each. Therefore, an important step is normalization, which places $\hat{Y}_{\text{init}}$ and $\hat{Y}_{\text{rerank}}$ on the same scale to better merge the ranking results. We use $\hat{P}_{\text{init}}$ to represent the interaction probability between users and products, and $\hat{Y}_{\text{init}}$ is also obtained from $\hat{P}_{\text{init}}$. Similarly, $\hat{P}_{\text{rerank}}$ is also used to obtain $\hat{Y}_{\text{rerank}}$. The process of normalization can be formalized as follows,

$$\begin{cases} \hat{P}_{\text{init}}^{\text{norm}} = \dfrac{\hat{P}_{\text{init}} - \min(\hat{P}_{\text{init}})}{\max(\hat{P}_{\text{init}}) - \min(\hat{P}_{\text{init}})}, \\ \hat{P}_{\text{rerank}}^{\text{norm}} = \dfrac{\hat{P}_{\text{rerank}} - \min(\hat{P}_{\text{rerank}})}{\max(\hat{P}_{\text{rerank}}) - \min(\hat{P}_{\text{rerank}})}. \end{cases} \tag{9}$$

After normalization and filtering, we adjust the hyperparameter α and β to merge $\hat{Y}_{\text{init}}$ and $\hat{Y}_{\text{rerank}}$,

$$\begin{cases} \hat{P} = \alpha \cdot \hat{P}_{\text{init}}^{norm} + (1 - \alpha) \cdot \hat{P}_{\text{rerank}}^{norm}, \\ \hat{Y} = \text{sort}(S, \hat{P}). \end{cases} \tag{10}$$

3.2.4 Collaborative-Decision Request. After collaborative training and collaborative inference, the LSC4Rec has formed a complete framework. This framework can compensate for the performance decline due to LLM's difficulty in obtaining real-time data as much as possible through SRM, making LSC4Rec more effective than using LLM4Rec or SRM4Rec independently in this system. However, a noteworthy point is that when LSC4Rec should invoke LLM to provide new inference results based on real-time data has not been addressed. To fill this gap, we designed the Collaborative-Decision Request feature for LSC4Rec.

We utilize the positional difference in rankings of the same element by $\hat{y}_{\text{init}}$ and $\hat{y}_{\text{rerank}}$ to calculate a inconsistency score c. Assuming $\text{pos}_{\text{p}}(\text{q})$ represents the position of element q in list p, the inconsistency score c can be formalized as follows,

$$c = \frac{1}{n} \sum\nolimits_{u \in \hat{y}_{\text{init}} \cap \hat{y}_{\text{rerank}}} |\text{pos}_{\hat{Y}_{\text{init}}}(\text{u}) - \text{pos}_{\hat{Y}_{\text{rerank}}}(\text{u})|, \tag{11}$$

where $\hat{y}_{\text{init}}$ and $\hat{y}_{\text{rerank}}$ In the above formula, $\hat{y}_{\text{init}} \cap \hat{y}_{\text{rerank}} = \hat{y}_{\text{init}} = \hat{y}_{\text{rerank}}$. This is because the elements in lists $\hat{y}_{\text{init}}$ and $\hat{y}_{\text{rerank}}$ are exactly the same, only their order differs.

$$\text{Request} = \mathbb{1}(\text{c} \geq \text{Threshold}). \tag{12}$$

In the equation above, $\mathbb{1}(\cdot)$ is the indicator function. To get the threshold, we need to collect user data for a period of time, then get the inconsistency c corresponding to these data on the cloud and sort them, and then set the threshold according to the load of the cloud server. That is, the threshold can be obtained during the training on the training set. For example, if the load of the cloud server needs to be reduced by 90%, that is, when the load is only 10% of the previous value, only the minimum 10% position value needs to be sent to each device as the threshold. During the inference process, each device determines whether it needs to upload real-time data to the LLM for inference, based on formulas 11 and 12. This is done to ensure the most optimal LLM invocation under any device-cloud communication resources and LLM invocation resources.

4 Experiments

4.1 Experimental Setup

4.1.1 Datasets. We evaluated our method on three widely used datasets, which are all collected from an e-commerce platform Amazon[1] and cover various product categories such as books, electronics, home goods, and more, they usually include user reviews, ratings, product descriptions, prices, and other information. We also evaluated our method on Yelp[2] which contains details about businesses, user reviews, ratings, business categories, addresses, and more. In accordance with convention, all user-item pairs in the datasets were considered as positive samples. For the training and test sets, user-item pairs that did not exist in the datasets were sampled as negative samples [11, 17]. During testing, we rank on a sample set consisting of all items. In these datasets, we use a user's last behavior for testing, the second to last behavior for validation, and the remaining behaviors for training.

4.1.2 Baselines. We used the following SRMs and LLMs for the experiments.

- **SRMs**. *DIN* [64], *GRU4Rec* [12], and *SASRec* [15] are three highly prevalent SRMs in both academic research and the industry. They each incorporate different techniques, such as GRU (Gated Recurrent Unit), Attention, and Self-Attention, to enhance the recommendation process. One point to note is that although models like BERT4Rec and E4SRec achieve better performance, their significantly larger number of model parameters results in far greater resource consumption compared to the aforementioned lightweight SRMs. Therefore, we

[1] https://jmcauley.ucsd.edu/data/amazon/
[2] https://www.yelp.com/dataset

Table 1: Performance comparison of baselines and LSC4Rec.

Dataset	Model	Metric								
		NDCG@5	NDCG@10	NDCG@20	HR@5	HR@10	HR@20	Precision@5	Precision@10	Precision@20
Beauty	DIN (RT)	0.0079	0.0106	0.0135	0.0131	0.0226	0.0311	0.0026	0.0022	0.0018
	GRU4Rec (RT)	0.0081	0.0105	0.0129	0.0136	0.0212	0.0308	0.0026	0.0022	0.0017
	SASRec (RT)	0.0066	0.0096	0.0127	0.0111	0.0207	0.0310	0.0022	0.0021	0.0018
	P5 (RT)	0.0227	0.0257	0.0289	0.0305	0.0400	0.0525	0.0061	0.0040	0.0026
	DIN (NRT)	0.0017	0.0026	0.0042	0.0021	0.0039	0.0083	0.0014	0.0014	0.0015
	GRU4Rec (NRT)	0.0017	0.0023	0.0031	0.0020	0.0033	0.0054	0.0014	0.0012	0.0010
	SASRec (NRT)	0.0014	0.0021	0.0032	0.0018	0.0033	0.0060	0.0013	0.0011	0.0011
	P5 (NRT)	0.0087	0.0108	0.0136	0.0132	0.0200	0.0310	0.0027	0.0020	0.0016
	Ours (P5+SASRec)	**0.0094**	**0.0126**	**0.0154**	**0.0150**	**0.0248**	**0.0361**	**0.0030**	**0.0025**	**0.0018**
	Improve	8.41%	16.16%	13.45%	13.14%	23.91%	16.59%	13.21%	24.00%	16.77%
Toys	DIN (RT)	0.0046	0.0063	0.0081	0.0076	0.0128	0.0202	0.0015	0.0013	0.0010
	GRU4Rec (RT)	0.0050	0.0073	0.0098	0.0085	0.0156	0.0253	0.0017	0.0016	0.0013
	SASRec (RT)	0.0052	0.0073	0.0101	0.0080	0.0148	0.0259	0.0016	0.0015	0.0013
	P5 (RT)	0.0187	0.0204	0.0221	0.0237	0.0291	0.0358	0.0047	0.0029	0.0018
	DIN (NRT)	0.0007	0.0010	0.0017	0.0010	0.0016	0.0035	0.0006	0.0005	0.0006
	GRU4Rec (NRT)	0.0009	0.0015	0.0020	0.0011	0.0023	0.0037	0.0008	0.0008	0.0007
	SASRec (NRT)	0.0015	0.0020	0.0026	0.0019	0.0030	0.0046	0.0012	0.0010	0.0008
	P5 (NRT)	0.0066	0.0078	0.0090	0.0096	0.0133	0.0183	0.0019	0.0013	0.0009
	Ours (P5+SASRec)	**0.0066**	**0.0084**	**0.0096**	**0.0105**	**0.0158**	**0.0209**	**0.0021**	**0.0016**	**0.0010**
	Improve	0.46%	7.33%	6.76%	8.62%	18.52%	13.74%	8.29%	18.80%	13.04%
Yelp	DIN (RT)	0.0067	0.0094	0.0126	0.0108	0.0191	0.0318	0.0022	0.0019	0.0016
	GRU4Rec (RT)	0.0045	0.0063	0.0084	0.0075	0.0131	0.0217	0.0015	0.0013	0.0011
	SASRec (RT)	0.0056	0.0079	0.0110	0.0087	0.0159	0.0281	0.0017	0.0016	0.0014
	P5 (RT)	0.0226	0.0253	0.0276	0.0299	0.0382	0.0471	0.0060	0.0038	0.0024
	DIN (NRT)	0.0033	0.0044	0.0057	0.0040	0.0062	0.0096	0.0026	<u>0.0020</u>	<u>0.0014</u>
	GRU4Rec (NRT)	0.0017	0.0026	0.0036	0.0022	0.0041	0.0065	0.0013	0.0013	0.0011
	SASRec (NRT)	0.0021	0.0030	0.0039	0.0024	0.0044	0.0066	0.0016	0.0015	0.0012
	P5 (NRT)	0.0093	0.0112	0.0132	0.0136	0.0196	0.0272	0.0027	0.0020	0.0014
	Ours (P5+SASRec)	**0.0095**	**0.0116**	**0.0144**	**0.0145**	**0.0213**	**0.0323**	**0.0029**	**0.0021**	**0.0016**
	Improve	1.72%	3.65%	9.50%	7.00%	8.88%	18.85%	7.01%	8.67%	19.12%

Table 2: The impact of choosing different LLMs on performance.

Dataset	LLM	Metric								
		NDCG@5	NDCG@10	NDCG@20	HR@5	HR@10	HR@20	Precision@5	Precision@10	Precision@20
Beauty	POD (NRT)	0.0079	0.0111	0.0134	0.0139	0.0239	0.0327	0.0028	0.0024	0.0016
	Ours(POD+SASRec)	**0.0091**	**0.0125**	**0.0150**	**0.0150**	**0.0256**	**0.0356**	**0.0030**	**0.0026**	**0.0018**
	Improv	15.59%	12.41%	12.27%	8.37%	7.29%	8.62%	8.30%	7.11%	8.54%

Table 3: The impact of choosing different SRMs on performance.

Dataset	SRM	Metric								
		NDCG@5	NDCG@10	NDCG@20	HR@5	HR@10	HR@20	Precision@5	Precision@10	Precision@20
Beauty	DIN	0.0091	0.0118	0.0147	0.0145	0.0230	0.0343	0.0029	0.0023	0.0017
	GRU4Rec	0.0092	0.0124	0.0151	0.0146	0.0247	0.0353	0.0029	0.0024	0.0017
	SASRec	**0.0094**	**0.0126**	**0.0154**	**0.0150**	**0.0248**	**0.0361**	**0.0030**	**0.0025**	**0.0018**

choose not to use them as models on the device.

- **LLMs**. *P5* [11] and *POD* [17] are two well-known LLM paradigms for recommendation, aiming to transform the recommendation task into textual format and use the text as input to the LLM to obtain recommendation results.

In the subsequent experimental results, unless otherwise specified, LLM defaults to P5 and SRM defaults to SASRec.

4.1.3 Evaluation metrics. In the experiments, we use the widely adopted NDCG, HitRate(HR) and Precision as the metrics to evaluate model performance. The details of the metrics are in Appendix.

4.2 Experimental Results

We use bold to denote the **best** value and underline to denote the <u>second-best</u> value (if applicable). Typically, we conduct such comparisons for each dataset. However, in some cases, such as in Figure 5, we compare values for each Request Frequency. In the experimental analysis section, when results across all datasets would take up significant space and consistent conclusions are drawn for each dataset, we only present the results for one dataset (e.g., Figure 5) to save space for more critical content.

4.2.1 Overall performance. Table 1 presents the performance comparison between LSC4Rec, SRMs, and LLMs under both real-time (RT) and near-real-time (NRT) data settings. Compared to real-time

Table 4: The ablation studies of the collaborative training.

Dataset	Training Method		Metric								
	Collaborative Training	Adaptive Re-training	NDCG@5	NDCG@10	NDCG@20	HR@5	HR@10	HR@20	Precision@5	Precision@10	Precision@20
Beauty	✗	✗	0.0073	0.0101	0.0130	0.0116	0.0204	0.0322	0.0023	0.0020	0.0016
	✓	✗	0.0089	0.0113	0.0138	0.0143	0.0216	0.0318	0.0029	0.0022	0.0016
	✗	✓	0.0093	0.0122	0.0150	0.0145	0.0236	0.0347	0.0029	0.0024	0.0017
	✓	✓	**0.0094**	**0.0126**	**0.0154**	**0.0150**	**0.0248**	**0.0361**	**0.0030**	**0.0025**	**0.0018**
Toys	✗	✗	0.0047	0.0065	0.0082	0.0074	0.0130	0.0194	0.0015	0.0013	0.0009
	✓	✗	0.0062	0.0074	0.0087	0.0098	0.0136	0.0189	0.0020	0.0014	0.0009
	✗	✓	0.0064	0.0080	0.0095	0.0099	0.0146	0.0208	0.0020	0.0015	**0.0010**
	✓	✓	**0.0066**	**0.0084**	**0.0096**	**0.0105**	**0.0158**	**0.0209**	**0.0021**	**0.0016**	**0.0010**
Yelp	✗	✗	**0.0068**	0.0090	0.0120	**0.0104**	0.0173	0.0291	0.0021	0.0017	0.0014
	✓	✗	0.0089	0.0110	0.0130	0.0137	0.0202	0.0279	0.0027	0.0019	0.0014
	✗	✓	0.0083	0.0109	0.0136	0.0131	0.0209	0.0317	0.0026	0.0020	0.0015
	✓	✓	**0.0095**	**0.0116**	**0.0144**	**0.0145**	**0.0213**	**0.0323**	**0.0029**	**0.0021**	**0.0016**

Table 5: The ablation studies of the collaborative inference.

Dataset	Inference Method			Metric								
	LLM	SRM	Result Fusion	NDCG@5	NDCG@10	NDCG@20	HR@5	HR@10	HR@20	Precision@5	Precision@10	Precision@20
Beauty	✓	✗	✗	0.0087	0.0108	0.0136	0.0132	0.0200	0.0310	0.0027	0.0020	0.0016
	✗	✓	✗	0.0014	0.0021	0.0032	0.0018	0.0033	0.0060	0.0013	0.0011	0.0011
	✓	✓	✗	**0.0094**	0.0122	0.0144	**0.0150**	0.0238	0.0329	0.0030	0.0024	0.0016
	✓	✓	✓	**0.0094**	**0.0126**	**0.0154**	**0.0150**	**0.0248**	**0.0361**	**0.0030**	**0.0025**	**0.0018**
Toys	✓	✗	✗	0.0087	0.0108	0.0136	0.0132	0.0200	0.0310	0.0027	0.0020	0.0016
	✗	✓	✗	0.0015	0.0020	0.0026	0.0019	0.0030	0.0046	0.0012	0.0010	0.0008
	✓	✓	✗	0.0059	0.0074	0.0089	0.0092	0.0138	0.0198	0.0018	0.0014	0.0010
	✓	✓	✓	**0.0094**	**0.0126**	**0.0154**	**0.0150**	**0.0248**	**0.0361**	**0.0030**	**0.0025**	**0.0018**
Yelp	✓	✗	✗	0.0087	0.0108	0.0136	0.0132	0.0200	0.0310	0.0027	0.0020	0.0016
	✗	✓	✗	0.0041	0.0058	0.0083	0.0066	0.0120	0.0218	0.0013	0.0012	0.0011
	✓	✓	✗	0.0083	0.0108	0.0134	0.0127	0.0205	0.0311	0.0025	0.0021	0.0016
	✓	✓	✓	**0.0094**	**0.0126**	**0.0154**	**0.0150**	**0.0248**	**0.0361**	**0.0030**	**0.0025**	**0.0018**

data, near-real-time data for each user excludes the most recent two interactions. As our study focuses on sequential recommendation tasks, we trained and tested LLMs exclusively on sequential recommendation datasets, excluding other types of datasets such as rating prediction. For LSC4Rec, LLMs first receive the same two-click-delayed near-real-time data to generate a candidate list of length 50. This candidate list is then re-ranked by SRMs using real-time data. In our setup, LLMs are instantiated as P5 or POD, and SRMs are set to SASRec (the evaluation of collaboration with other SRMs can be found in Table 3). From Table 1, we can draw the following conclusions:

- **Necessity of LLMs:** Under various conditions of accessing real-time data, the performance of LLMs is superior to that of SRMs in most cases, highlighting the necessity of utilizing on-cloud LLMs.
- **Necessity of SRMs:** Although the performance of SRM based on real-time data sometimes surpasses that of LSC4Rec in some metrics, achieving this in practical applications is quite challenging. This is because SRMs often cannot access real-time data (if deployed on the cloud) and all of the item embeddings (if deployed on the device). Nevertheless, the high performance of SRM with real-time data highlights the unique advantages that on-device SRMs bring to LSC4Rec, which LLMs lack.
- **Impact of Real-Time Data Absence on LLMs and SRMs:** Compared to real-time data, the performance of both LLMs and SRMs drops significantly when using near-real-time data delayed by two user interactions. This underscores the negative impact of lacking access to real-time user behavior on models, especially on cloud-based LLMs.
- **Effectiveness of LSC4Rec:** Our LSC4Rec method achieves significant performance improvements across all three datasets. Specifically, when LLM is set to P5 and SRM to SASRec, LSC4Rec achieves average improvements of 16.18%, 10.62%, and 9.38% on the Beauty, Toys, and Yelp datasets, respectively. These results indicate that our method effectively mitigates the performance degradation of LLMs caused by the lack of access to real-time user behavior.

4.2.2 The impact of choosing different SRMs on performance. To observe the impact of small model selection on LSC4Rec, we conducted experiments using various small models. As shown in Table 2, the experimental results indicate that: LSC4Rec achieves significant performance improvements on POD, indicating that the LSC4Rec framework can effectively mitigate the performance degradation of various LLMs caused by their inability to access real-time data.

4.2.3 The impact of choosing different SRMs on performance. To observe the impact of small model selection on LSC4Rec, we conducted experiments based on various small models. As shown in Table 3, the experimental results indicate that:

Table 6: The ablation studies of the collaborative-decision request.

Dataset	Decision Method	Request Frequency	Metric								
			NDCG@5	NDCG@10	NDCG@20	HR@5	HR@10	HR@20	Precision@5	Precision@10	Precision@20
Beauty	Random	5%	0.0091	0.0114	0.0142	0.0138	0.0209	0.0321	0.0028	0.0021	0.0016
	Inconsistency	5%	**0.0125**	**0.0146**	**0.0173**	**0.0173**	**0.0240**	**0.0349**	**0.0035**	**0.0024**	**0.0017**
	Improvement		36.66%	28.14%	21.89%	25.71%	14.80%	8.63%	25.82%	14.83%	8.07%
	Random	10%	0.0097	0.0120	0.0149	0.0145	0.0219	0.0332	0.0029	0.0022	0.0017
	Inconsistency	10%	**0.0134**	**0.0156**	**0.0183**	**0.0184**	**0.0254**	**0.0363**	**0.0037**	**0.0025**	**0.0018**
	Improvement		37.91%	29.56%	23.13%	26.50%	15.91%	9.31%	26.46%	15.98%	9.04%
	Random	20%	0.0104	0.0127	0.0157	0.0153	0.0227	0.0344	0.0031	0.0023	0.0017
	Inconsistency	20%	**0.0140**	**0.0162**	**0.0190**	**0.0192**	**0.0263**	**0.0373**	**0.0038**	**0.0026**	**0.0019**
	Improvement		34.72%	27.42%	21.19%	25.44%	15.54%	8.32%	25.49%	15.42%	8.14%
	Random	40%	0.0137	0.0162	0.0192	0.0195	0.0274	0.0393	0.0039	0.0027	0.0020
	Inconsistency	40%	**0.0164**	**0.0189**	**0.0219**	**0.0225**	**0.0305**	**0.0423**	**0.0045**	**0.0031**	**0.0021**
	Improvement		19.82%	16.92%	14.18%	15.89%	11.58%	7.74%	15.94%	11.31%	8.16%

- The choice of small models indeed has some influence on LSC4Rec, which can be attributed to differences in the ability of different SRMs to interpret user preferences. Overall, SASRec provides the most significant improvement for LSC4Rec. We conducted analyses on the Beauty, Toys, and Yelp datasets. Although SASRec does not outperform GRU4Rec and DIN across all metrics on all datasets, it generally provides the most significant improvements to LSC4Rec. Therefore, to save space, we present only the results on the Beauty dataset.
- Within the LSC4Rec framework, regardless of the SRM used, the performance is superior to that of LLMs without access to real-time data.

4.3 Ablation Study

For simplicity, we use "**w.**" to represent "with" and "**w/o.**" to represent "without". In these experiments, the SRM on the device uses the SASRec model.

4.3.1 The impact of collaborative training. As shown in Table 4, we analyze the effectiveness of each component in collaborative training via ablation study. There are four rows for each dataset, corresponding to four types of ablations, we provide the following explanations for them:

- **w/o.** Collaborative Training and **w/o.** Adaptive Re-training (Row.1) indicates that both the LLMs and SRMs are trained independently based on historical user data.
- **w.** Collaborative Training and **w/o.** Adaptive Re-training (Row.2) indicates that after both the LLMs and SRMs are trained independently based on historical user data, the SRM then learns to re-rank the candidate list generated by the LLM.
- **w/o.** Collaborative Training and **w.** Adaptive Re-training (Row.3) indicates that after both the LLMs and SRMs are trained independently based on historical user data, the SRM is then re-trained on the device based on the user's real-time data.
- **w.** Collaborative Training and **w.** Adaptive Re-training (Row.4) indicates that after both the LLMs and SRMs are trained independently based on historical user data, the SRM learns to re-rank the candidate list generated by the LLM. Then the SRM is re-trained on the device based on the user's real-time data.

The experimental results show that with cooperative training and adaptive re-training performs the best, followed by with adaptive re-training and without cooperative training. The worst case occurs when neither is used, and the gap compared to the other three cases is significant. This indicates that on-device SRM cannot perform well within the LSC4Rec framework solely through independent training. These results demonstrate the effectiveness of cooperative training and adaptive re-training.

4.3.2 The impact of collaborative Inference. To analyze the effectiveness of each component in our collaborative inference strategy, we conducted ablation experiments on the inference strategy. As shown in Table 5, each dataset has four rows of experimental results, corresponding to four types of ablations. For the four rows of data for each dataset, we provide the following explanations:

- **w.** LLM, **w/o.** SRM, **w/o.** Result Fusion (Row.1) indicates that only use on-cloud LLM to recall the candidate list, the initial ranking results can be regarded as the full-ranking results based on the less real-time data.
- **w/o.** LLM, **w.** SRM, **w/o.** Result Fusion (Row.2) indicates that only use on-device SRM to do ranking.
- **w.** LLM, **w.** SRM, **w/o.** Result Fusion (Row.3) indicates first to let on-cloud LLM recall a candidate list, then on-device SRM do rerank the candidate list based on the real-time
- **w.** LLM, **w.** SRM, **w.** Result Fusion (Row.4) indicates first to let on-cloud LLM recall a candidate list and initial ranking resuls, then on-device SRM do rerank the candidate list based on the real-time, further, reranking and initial ranking ard fused as the final ranking results.

The experimental results show that the collaborative inference and result fusion of LLMs and SRMs perform the best, followed by collaborative inference of LLMs and SRMs without result fusion. Using either the LLM or the SRM alone results in poorer performance. The above results demonstrate the effectiveness of our designed collaborative inference and result fusion.

4.3.3 The impact of collaborative-decision request. As shown in Table 6, we analyze the collaborative-decision request.

- **Random** (Row.1) indicates that choosing some users to update the candidate list and corresponded initial ranking results randomly.

Table 7: The impact of length of candidate list on performance

Dataset	Candidate Length	Metrics								
		NDCG@5	NDCG@10	NDCG@20	HR@5	HR@10	HR@20	Precision@5	Precision@10	Precision@20
Beauty	20	**0.0097**	0.0124	0.0143	**0.0152**	0.0238	0.0312	**0.0030**	0.0024	0.0016
	50	0.0094	**0.0126**	**0.0154**	0.0150	**0.0248**	**0.0361**	**0.0030**	**0.0025**	**0.0018**
Toys	20	**0.0067**	0.0080	0.0090	**0.0106**	0.0148	0.0186	**0.0021**	0.0015	0.0009
	50	0.0066	**0.0084**	**0.0096**	0.0105	**0.0158**	**0.0209**	**0.0021**	**0.0016**	**0.0010**
Yelp	20	**0.0099**	**0.0121**	0.0136	**0.0148**	**0.0214**	0.0276	**0.0030**	**0.0021**	0.0014
	50	0.0095	0.0116	**0.0144**	0.0145	0.0213	**0.0323**	0.0029	**0.0021**	**0.0016**

- **Inconsistency** (Row.2) indicates that choosing some users to update the candidate list and corresponded initial ranking results based on the ranking results of the LLM and SRM. High inconsistency can get more priority to update.

The experimental results indicate that our strategy of selecting user data for generating new candidate lists and initial ranking results with LLM, based on the inconsistency of ranking between LLM and SRM, achieves an average improvement of about 25% compared to random selection. This is because our strategy tends to prioritize updates of candidate lists and initial rankings with the highest benefit, while random updating lacks the capability to calculate the benefits of updates. At the same time, we can observe that the peak of improvement occurs at a request frequency of 10%, and then as the frequency increases, the magnitude of improvement diminishes. This is because when the request frequency increases (e.g., to 20%, 40%), our strategy has already updated the parts with the highest benefits, while random updates mix high and low benefit updates, leading to a decrease in improvement. In summary, the above results all demonstrate that our strategy effectively selects the users with the highest benefits for updating candidate lists and initial rankings.

4.4 In-Depth Analysis

4.4.1 The impact of length of candidate list. In Table 7, we analyze the impact of the length of the LLM's recall list on performance. When the length of the candidate list is 20 and 50, the range of performance variation is not significant, which indicates that LSC4Rec is not very sensitive to this hyperparameter of the recall list's length. Additionally, shorter candidate lists tend to perform better on stricter metrics (e.g., @5), while longer candidate lists show advantages on more lenient metrics (e.g., @20). The advantage of a longer candidate list lies in its higher likelihood of including the ground-truth item, but the downside is the greater challenge for the SRM to re-rank longer candidate lists effectively. On a relatively balanced metric (e.g., @10), the advantages and disadvantages of longer candidate lists are approximately equal.

4.4.2 Analysis of resource consumption. To validate the resource consumption of our method, we analyzed the resource usage of our approach in Table 8. Since the choice of either P5 or POD as the LLM does not affect the conclusion, we will use one of the models as an example. Here, we choose P5 to describe the conclusion. In terms of the number of parameters, if we uses P5 as LLM and SASRec as SRM, the total number of parameters is the sum of P5 and SASRec (on-device consumption is same as SASRec). In terms of training duration, since our method does not require real-time training of LLMs, the time consumption is reduced compared to training LLMs.

Table 8: Resource consumption comparison.

Model	#Parameter	Training Time (s/epoch)	Inference Time (s/sample)
On-device			
DIN	0.82M	38.50	0.00074
GRU4Rec	0.81M	38.30	0.00085
SASRec	1.62M	39.40	0.00143
On-cloud			
P5	60.75M	897.83	0.10082
POD	60.77M	2277.16	0.32563

In terms of inference duration, because our method does not need to call the LLM at every moment, and often does not need to call the LLM at all, the time consumption is significantly reduced compared to the LLM, and has not increased much compared to the SRM, which does not affect the real-time nature of the recommendations. Overall, our method leverages the advantages of LLMs with low resource consumption to achieve better performance.

5 Conclusion

The proposed Device-Cloud Collaborative Framework for LLM and SRM (LSC4Rec) effectively combines the strengths of large language models and small recommendation models. By leveraging collaborative training, collaborative inference, and intelligent request strategies, LSC4Rec addresses the limitations of LLMs in capturing real-time user preferences while optimizing the utilization of cloud and edge computing resources. Extensive experimental results validate the effectiveness of each strategy, highlighting the framework's potential for practical applications. Future work will focus on further refining the collaboration mechanisms to enhance recommendation performance and user satisfaction.

ACKNOWLEDGEMENTS

This work was supported by 2030 National Science and Technology Major Project (2022ZD0119100), Scientific Research Fund of Zhejiang Provincial Education Department (Y202353679), National Natural Science Foundation of China (No. 62402429, 62376243, 62441605, 62037001), the Key Research and Development Program of Zhejiang Province (No. 2024C03270), ZJU Kunpeng&Ascend Center of Excellence, the Key Research and Development Projects in Zhejiang Province (No.2024C01106), Zhejiang University Education Foundation Qizhen Scholar Foundation, the Starry Night Science Fund at Shanghai Institute for Advanced Study (Zhejiang University). This work was also supported by Ant group.

References

[1] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation. (2023), 1007–1014.

[2] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. ACM, 1007–1014.

[3] Yongjun Chen, Zhiwei Liu, Jia Li, Julian McAuley, and Caiming Xiong. 2022. Intent contrastive learning for sequential recommendation. In *Proceedings of the ACM Web Conference 2022*. 2172–2182.

[4] Zhengyu Chen, Donglin Wang, and Shiqian Yin. 2021. Improving cold-start recommendation via multi-prior meta-learning. In *Advances in Information Retrieval: 43rd European Conference on IR Research, ECIR 2021, Virtual Event, March 28–April 1, 2021, Proceedings, Part II 43*. Springer, 249–256.

[5] Zhengyu Chen, Ziqing Xu, and Donglin Wang. 2021. Deep transfer tensor decomposition with orthogonal constraint for recommender systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 4010–4018.

[6] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering ChatGPT's Capabilities in Recommender Systems. In *RecSys*. ACM, 1126–1132.

[7] Kairui Fu, Qiaowei Miao, Shengyu Zhang, Kun Kuang, and Fei Wu. 2023. End-to-End Optimization of Quantization-Based Structure Learning and Interventional Next-Item Recommendation. In *CAAI International Conference on Artificial Intelligence*. Springer, 415–429.

[8] Kairui Fu, Shengyu Zhang, Zheqi Lv, Jingyuan Chen, and Jiwei Li. 2024. DIET: Customized Slimming for Incompatible Networks in Sequential Recommendation. In *KDD*. ACM, 816–826.

[9] Chongming Gao, Shiqi Wang, Shijun Li, Jiawei Chen, Xiangnan He, Wenqiang Lei, Biao Li, Yuan Zhang, and Peng Jiang. 2023. CIRS: Bursting Filter Bubbles by Counterfactual Interactive Recommender System. *ACM Transactions on Information Systems (TOIS)* 42, 1, Article 14 (aug 2023), 27 pages. doi:10.1145/3594871

[10] Yunfan Gao, Tao Sheng, Youlin Xiang, Yun Xiong, Haofen Wang, and Jiawei Zhang. 2023. Chat-rec: Towards interactive and explainable llms-augmented recommender system. *arXiv preprint arXiv:2303.14524* (2023).

[11] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5). In *RecSys*. ACM, 299–315.

[12] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based recommendations with recurrent neural networks. *International Conference on Learning Representations 2016* (2016).

[13] Jianchao Ji, Zelong Li, Shuyuan Xu, Wenyue Hua, Yingqiang Ge, Juntao Tan, and Yongfeng Zhang. 2023. Genrec: Large language model for generative recommendation. *arXiv e-prints* (2023), arXiv–2307.

[14] Wei Ji, Xiangyan Liu, An Zhang, Yinwei Wei, Yongxin Ni, and Xiang Wang. 2023. Online distillation-enhanced multi-modal transformer for sequential recommendation. In *Proceedings of the 31st ACM International Conference on Multimedia*. 955–965.

[15] Wang-Cheng Kang and Julian J. McAuley. 2018. Self-Attentive Sequential Recommendation. In *IEEE International Conference on Data Mining, ICDM 2018, Singapore, November 17-20, 2018*. IEEE Computer Society, 197–206. doi:10.1109/ICDM.2018.00035

[16] Heng Li, Zhiyuan Yao, Bang Wu, Cuiying Gao, Teng Xu, Wei Yuan, and Xiapu Luo. 2025. Automated Malware Assembly Line: Uniting Piggybacking and Adversarial Example in Android Malware Generation. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24 - February 28, 2025*. The Internet Society.

[17] Lei Li, Yongfeng Zhang, and Li Chen. 2023. Prompt Distillation for Efficient LLM-based Recommendation. In *CIKM*. ACM, 1348–1357.

[18] Wentong Li, Yuqian Yuan, Jian Liu, Dongqi Tang, Song Wang, Jie Qin, Jianke Zhu, and Lei Zhang. 2024. Tokenpacker: Efficient visual projector for multimodal llm. *arXiv preprint arXiv:2407.02392* (2024).

[19] Yangfan Li, Kenli Li, Wei Wei, Tianyi Zhou, and Cen Chen. 2022. CoRec: an efficient internet behavior-based recommendation framework with edge-cloud collaboration on deep convolution neural networks. *ACM Transactions on Sensor Networks* 19, 2 (2022), 1–28.

[20] Xinting Liao, Weiming Liu, Xiaolin Zheng, Binhui Yao, and Chaochao Chen. 2023. Ppgencdr: A stable and robust framework for privacy-preserving cross-domain recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 4453–4461.

[21] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050* (2023).

[22] Xinyu Lin, Wenjie Wang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Bridging items and language: A transition paradigm for large language model-based recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1816–1826.

[23] Xinyu Lin, Chaoqun Yang, Wenjie Wang, Yongqi Li, Cunxiao Du, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Efficient Inference for Large Language Model-based Generative Recommendation. *CoRR* abs/2410.05165 (2024).

[24] Junling Liu, Chao Liu, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is ChatGPT a Good Recommender? A Preliminary Study. *CoRR* abs/2304.10149 (2023).

[25] Kai Liu, Ze Chen, Zhihang Fu, Rongxin Jiang, Fan Zhou, Yaowu Chen, Yue Wu, and Jieping Ye. 2024. Structure-aware Domain Knowledge Injection for Large Language Models. *arXiv preprint arXiv:2407.16724* (2024).

[26] Kai Liu, Zhihang Fu, Chao Chen, Wei Zhang, Rongxin Jiang, Fan Zhou, Yaowu Chen, Yue Wu, and Jieping Ye. 2024. Enhancing LLM's Cognition via Structurization. *Advances in Neural Information Processing Systems* 38 (2024).

[27] Weiming Liu, Chaochao Chen, Xinting Liao, Mengling Hu, Yanchao Tan, Fan Wang, Xiaolin Zheng, and Yew Soon Ong. 2024. Learning Accurate and Bidirectional Transformation via Dynamic Embedding Transportation for Cross-Domain Recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8815–8823.

[28] Weiming Liu, Xiaolin Zheng, Chaochao Chen, Jiajie Su, Xinting Liao, Mengling Hu, and Yanchao Tan. 2023. Joint internal multi-interest exploration and external domain alignment for cross domain sequential recommendation. In *Proceedings of the ACM Web Conference 2023*. 383–394.

[29] Jing Long, Guanhua Ye, Tong Chen, Yang Wang, Meng Wang, and Hongzhi Yin. 2024. Diffusion-Based Cloud-Edge-Device Collaborative Learning for Next POI Recommendations. In *KDD*. ACM, 2026–2036.

[30] Jing Long, Guanhua Ye, Tong Chen, Yang Wang, Meng Wang, and Hongzhi Yin. 2024. Diffusion-based cloud-edge-device collaborative learning for next POI recommendations. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2026–2036.

[31] Sichun Luo, Bowei He, Haohan Zhao, Yinya Huang, Aojun Zhou, Zongpeng Li, Yuanzhang Xiao, Mingjie Zhan, and Linqi Song. 2023. RecRanker: Instruction Tuning Large Language Model as Ranker for Top-k Recommendation. *CoRR* abs/2312.16018 (2023).

[32] Zheqi Lv, Shaoxuan He, Tianyu Zhan, Shengyu Zhang, Wenqiao Zhang, Jingyuan Chen, Zhou Zhao, and Fei Wu. 2024. Semantic Codebook Learning for Dynamic Recommendation Models. In *ACM Multimedia*. ACM, 9611–9620.

[33] Zheqi Lv, Wenqiao Zhang, Zhengyu Chen, Shengyu Zhang, and Kun Kuang. 2024. Intelligent Model Update Strategy for Sequential Recommendation. In *WWW*. ACM, 3117–3128.

[34] Zheqi Lv, Wenqiao Zhang, Shengyu Zhang, Kun Kuang, Feng Wang, Yongwei Wang, Zhengyu Chen, Tao Shen, Hongxia Yang, Beng Chin Ooi, and Fei Wu. 2023. DUET: A Tuning-Free Device-Cloud Collaborative Parameters Generation Framework for Efficient Device Model Generalization. In *WWW*. ACM, 3077–3085.

[35] Xufeng Qian, Yue Xu, Fuyu Lv, Shengyu Zhang, Ziwen Jiang, Qingwen Liu, Xiaoyi Zeng, Tat-Seng Chua, and Fei Wu. 2022. Intelligent request strategy design in recommender system. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3772–3782.

[36] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.

[37] Jiajie Su, Chaochao Chen, Weiming Liu, Fei Wu, Xiaolin Zheng, and Haoming Lyu. 2023. Enhancing hierarchy-aware graph networks with deep dual clustering for session-based recommendation. In *Proceedings of the ACM Web Conference 2023*. 165–176.

[38] Teng Sun, Chun Wang, Xuemeng Song, Fuli Feng, and Liqiang Nie. 2022. Response generation by jointly modeling personalized linguistic styles and emotions. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)* 18, 2 (2022), 1–20.

[39] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. (2023), 14918–14937.

[40] Yuchong Sun, Che Liu, Kun Zhou, Jinwen Huang, Ruihua Song, Wayne Xin Zhao, Fuzheng Zhang, Di Zhang, and Kun Gai. 2024. Parrot: Enhancing multi-turn instruction following for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9729–9750.

[41] Jiaxi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.

[42] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023). doi:10.48550/ARXIV.2302.13971 arXiv:2302.13971

[43] Jiawei Wang, Da Cao, Shaofei Lu, Zhanchang Ma, Junbin Xiao, and Tat-Seng Chua. 2024. Causal-driven Large Language Models with Faithful Reasoning for Knowledge Question Answering. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 4331–4340.

[44] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2023. A Survey on Large Language Models for Recommendation. *arXiv preprint arXiv:2305.19860* (2023).

[45] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, Hui Xiong, and Enhong Chen. 2024. A survey on large language models for recommendation. *World Wide Web (WWW)* 27, 5 (2024), 60.

[46] Shengqiong Wu, Hao Fei, Leigang Qu, Wei Ji, and Tat-Seng Chua. 2024. NExT-GPT: Any-to-Any Multimodal LLM. In *ICML*. OpenReview.net.

[47] Chen Xu, Wenjie Wang, Yuxin Li, Liang Pang, Jun Xu, and Tat-Seng Chua. 2024. A Study of Implicit Ranking Unfairness in Large Language Models. In *EMNLP (Findings)*. Association for Computational Linguistics, 7957–7970.

[48] Jiangchao Yao, Feng Wang, Xichen Ding, Shaohu Chen, Bo Han, Jingren Zhou, and Hongxia Yang. 2022. Device-cloud Collaborative Recommendation via Meta Controller. In *KDD*. ACM, 4353–4362.

[49] Jiangchao Yao, Feng Wang, Kunyang Jia, Bo Han, Jingren Zhou, and Hongxia Yang. 2021. Device-Cloud Collaborative Learning for Recommendation. In *KDD*. ACM, 3865–3874.

[50] Honglei Zhang, He Liu, Haoxuan Li, and Yidong Li. 2024. Transfr: Transferable federated recommendation with pre-trained language models. *arXiv:2402.01124* (2024).

[51] Jinghao Zhang, Qiang Liu, Shu Wu, and Liang Wang. 2023. Mining Stable Preferences: Adaptive Modality Decorrelation for Multimedia Recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 443–452.

[52] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as Instruction Following: A Large Language Model Empowered Recommendation Approach. *CoRR* abs/2305.07001 (2023). doi:10.48550/ARXIV.2305.07001 arXiv:2305.07001

[53] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as Instruction Following: A Large Language Model Empowered Recommendation Approach. *CoRR* abs/2305.07001 (2023).

[54] Jinghao Zhang, Yanqiao Zhu, Qiang Liu, Shu Wu, Shuhui Wang, and Liang Wang. 2021. Mining latent structures for multimedia recommendation. In *Proceedings of the 29th ACM international conference on multimedia*. 3872–3880.

[55] Jinghao Zhang, Yanqiao Zhu, Qiang Liu, Mengqi Zhang, Shu Wu, and Liang Wang. 2022. Latent structure mining with contrastive modality fusion for multimedia recommendation. *IEEE Transactions on Knowledge and Data Engineering* 35, 9 (2022), 9154–9167.

[56] Kepu Zhang, Teng Shi, Sunhao Dai, Xiao Zhang, Yinfeng Li, Jing Lu, Xiaoxue Zang, Yang Song, and Jun Xu. 2024. SAQRec: Aligning Recommender Systems to User Satisfaction via Questionnaire Feedback. In *CIKM*. ACM, 3165–3175.

[57] Shuo Zhang, Boci Peng, Xinping Zhao, Boren Hu, Yun Zhu, Yanjia Zeng, and Xuming Hu. 2024. LLaSA: Large Language and E-Commerce Shopping Assistant. In *Amazon KDD Cup 2024 Workshop*.

[58] Yang Zhang, Fuli Feng, Jizhi Zhang, Keqin Bao, Qifan Wang, and Xiangnan He. 2023. Collm: Integrating collaborative embeddings into large language models for recommendation. *arXiv preprint arXiv:2310.19488* (2023).

[59] Jujia Zhao, Wenjie Wang, Xinyu Lin, Leigang Qu, Jizhi Zhang, and Tat-Seng Chua. 2023. Popularity-aware Distributionally Robust Optimization for Recommendation System. In *CIKM*. ACM, 4967–4973.

[60] Jujia Zhao, Wang Wenjie, Yiyan Xu, Teng Sun, Fuli Feng, and Tat-Seng Chua. 2024. Denoising diffusion recommender model. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1370–1379.

[61] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, and Qing Li. 2024. Recommender Systems in the Era of Large Language Models (LLMs). *IEEE Trans. Knowl. Data Eng.* 36, 11 (2024), 6889–6907.

[62] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, and Qing Li. 2024. Recommender Systems in the Era of Large Language Models (LLMs). *IEEE Trans. Knowl. Data Eng.* 36, 11 (2024), 6889–6907.

[63] Xiaolin Zheng, Jiajie Su, Weiming Liu, and Chaochao Chen. 2022. DDGHM: Dual dynamic graph with hybrid metric training for cross-domain sequential recommendation. In *Proceedings of the 30th ACM International Conference on Multimedia*. 471–481.

[64] Guorui Zhou, Xiaoqiang Zhu, Chengru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *KDD*. ACM, 1059–1068.

[65] Didi Zhu, Zhongyisun Sun, Zexi Li, Tao Shen, Ke Yan, Shouhong Ding, Chao Wu, and Kun Kuang. 2024. Model Tailor: Mitigating Catastrophic Forgetting in Multi-modal Large Language Models. In *Forty-first International Conference on Machine Learning*.

[66] Yun Zhu, Yaoke Wang, Haizhou Shi, and Siliang Tang. 2024. Efficient Tuning and Inference for Large Language Models on Textual Graphs. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, Kate Larson (Ed.). International Joint Conferences on Artificial Intelligence Organization, 5734–5742. doi:10.24963/ijcai.2024/634 Main Track.

A Appendix

This is the Appendix for "Collaboration of Large Language Models and Small Recommendation Models for Device-Cloud Recommendation".

A.1 Supplementary Methodology

A.1.1 Pseudo code of LSCRec. Algorithm 1 shows the pseudo code of LSCRec. (x) represents that x is a intermediate variable.

Algorithm 1: Pseudo Code of LSC4Rec

Strategy 1: ▷ *Collaborative Training*

Target: Pretrained LLM $\mathcal{M}_L$, Pretrained SRM $\mathcal{M}_S \mapsto$ Updated SRM $\mathcal{M}_S$
Input: Historical data $\mathcal{D}_H$
Output: Prediction $\hat{Y}$

Strategy 2: ▷ *Collaborative Inference*

Target: Real-time data $\mathcal{D}_R \mapsto$ Prediction $\hat{Y}$
Input: Real-time data $\mathcal{D}_R$
Output: (Candidate Item Set $\mathcal{S}$), (Initial Ranking $\hat{Y}_{\text{init}}$ generated by $\mathcal{M}_L$), (Re-Ranking $\hat{Y}_{\text{rerank}}$ generated by $\mathcal{M}_S$), Prediction $\hat{Y}$

Strategy 3: ▷ *Collaborative-decision Request*

Target: Initial Ranking $\hat{Y}_{\text{init}}$, Re-Ranking $\hat{Y}_{\text{rerank}} \mapsto$ Inconsistency c
Input: Initial Ranking $\hat{Y}_{\text{init}}$, Re-Ranking $\hat{Y}_{\text{rerank}}$
Output: Inconsistency c

Overview: ▷ *Training Procedure*
Input: Historical data $\mathcal{D}_H$, Real-time data $\mathcal{D}_R$
Output: Prediction $\hat{y}$.
Initialization: Randomly initialize the $\mathcal{M}_L$ and $\mathcal{M}_S$
repeat
  if *$\mathcal{M}_L$ and $\mathcal{M}_S$ have not yet been well-trained* **then**
    Calculate loss as follows (see Eq.4 for the details),
    $\mathcal{L} = \sum_{u,v,s,y \in \mathcal{D}_H} l(y, \hat{y} = \mathcal{M}(u, v, s))$.
  end
until *Convergence*;
repeat
  if *$\mathcal{M}_S$ has not yet been well-trained* **then**
    Calculate loss as follows (see Eq.5∼7 for the details),
    $\mathcal{L} = \sum_{u,v,s,y \in \mathcal{D}_H} l(y, \hat{y} = \mathcal{M}_S(u, v, s)|S_{Aug})$.
  end
until *Convergence*;
return $\mathcal{M}_L$, $\mathcal{M}_S$.
Overview: ▷ *Inference Procedure*
Input: Real-time data $\mathcal{D}_R$
Output: Prediction $\hat{Y}$, Inconsistency c
S, $\hat{Y}_{\text{init}}$ predicted by LLM, $\hat{Y}_{\text{rerank}}$ predicted by SRM based on S.
c calculated based on $\hat{Y}_{\text{init}}$ and $\hat{Y}_{\text{rerank}}$ to help decide whether to request new S, $\hat{Y}_{\text{init}}$ from LLM.

A.2 Supplementary Experiments

A.2.1 Datasets. The statistics of the datasets used in the experiments is shown in Table 9.

Table 9: Statistics of Datasets.

Dataset	Beauty	Toys	Yelp
#User	22,363	19,412	30,431
#Item	12,101	11,924	20,033
#interaction	198,502	167,597	316,354
Density	0.00073352	0.00072406	0.00051893

A.2.2 Hyperparameters and Training Schedules. We summarize the hyperparameters and training schedules of the LLM and SRM used in the experiments. Table 11 shows the settings of the SRM training. Table 12 shows the setting of the collaborative training, collaborative inference, and collaborative-decision request.

Table 10: Hyperparameters of LLM training.

Dataset	LLM	Hyperparameter	Setting
Beauty Toys Yelp	P5	GPU	Tesla A100 (40GB)
		Batch Size	32
		Max Text Length	512
		Gen Max Length	64
		Learning Rate	1e-3
		Weight Decay	0.05(Beauty, Toys), 0.02(Yelp)
		Optimizer	Adamw
		Task	Sequential
		Negative Sampling Rate	1:99
		Beam Search	20/50
Beauty Toys Yelp	POD	GPU	Tesla A100 (40GB)
		Batch Size	64
		Learning Rate	5e-4
		Weight Decay	1e-2
		Optimizer	Adamw
		Task	Sequential
		Negative Sampling Rate	1:99
		Beam Search	20/50

Table 11: Hyperparameters of SRM training.

Dataset	SRM	Hyperparameter	Setting
Beauty Toys Yelp	DIN GRU4Rec SASRec	GPU	Tesla A100 (40GB)
		Optimizer	Adam
		Learning rate	1e-3
		Batch size	1024
		Sequence length	10
		the Dimension of Embedding	1×32
		the Amount of MLP	2

Table 12: Hyperparameters of collaborative training, collaborative inference, and collaborative-decision request.

Dataset	LLM	SRM	Hyperparameter	Setting
Beauty Toys Yelp	P5 POD	DIN GRU4Rec SASRec	Length of Candidate list	20/50
			Valid user behavior length	Dynamic
			User behavior length	10
			Sequence length	50
			Learning Rate	1e-3
			Dimension of embedding	1×32
			Optimizer	Adam